
Chapter 4 : **Functions**

FUNCTIONS

- Function is group of program statements that can act on data and return a value.
- Every C++ program has at least one function,
- When your program starts, main () is called automatically.
- main () might call other functions, some of which might call still others.

- The reason why we use functions is to aid modularization of a program.
- A function reduces program size.
- Any fragments of code that are needed frequently in a program are best candidates to be written as a function.

Function Declaration / Prototype Declaration

- As you can't use variables before declarations (telling the compiler what the variable is), you can't use function before telling the compiler what this function is.
- The common approach to declare functions is at the beginning of the program.
- The function declaration (function prototype) tells the compiler that later on in the program a function introduced in the declaration is going to be used.

Example

`void getnumber( );`//this is a function declaration.

- Void shows that the function doesn't have a return type.
- Function declaration is terminated with semicolon. If the function has arguments, then they should be indicated in the declaration.

Function Definition

A **function definition** consists of two parts: interface and body.

- The **interface** of a function (called its **prototype**) specifies how it may be used.

It consists of three entities:

- The function **name**. This is simply a unique identifier.
- The function **parameters** (also called its **signature**).

This is a set of zero or more typed identifiers used for passing values to and from the function.

- The function **return type**. This specifies the type of value the function returns.
- A function which returns nothing should have the return type void.
- The **body** of a function contains the computational steps (statements) that comprise the function.

Example

```
Void getnumber()
{
int x;
cout<< "Enter Number \n";
cin>>number;
cout<<"The number you entered is:"<<number;
}
```

The function body is placed in one place in memory. But it could be invoked in several places in the program.

Calling functions

- Calling functions is invoking it to execute.
- The function call involves the function name, followed by parentheses.
- The function call is similar to its declaration except that the **return type is not mentioned**.
- The call is terminated by semi colon.
- Executing the call statement causes the function to execute, i.e. control is transferred to the function, the statements in the function definition are executed and then control returns to the statement following the function call.

// the following program demonstrates what we have discussed so far
`#include<iostream>`

Using namespace std;

`void add ( );`

`void sub ( );`

`void main ( )`

`{ char ch;`

`cout <<" what do you want ? A: Add, S: subtract \n";`

`cin>>ch;`

`switch(ch) {`

`case 'A': // A for addition`

`add ( );`

`break;`

`case 'S' : // S for subtraction`

`sub ( );`

`break;`

`}}`

```

void add()
{
    int a,b,c;
    cout<<"enter two numbers \n";
    cin>> a>>b;
    c = a + b;
    cout <<" the sum is \t " << c;
}

void sub ( )
{
    int a,b,c;
    cout<<"Enter two numbers \n";
    cin>>a>>b;
    cout<<"the difference is \t " <<a-b;
}

```

Parameters and Arguments

- C++ supports two styles of parameters: *value and reference*.

A **value parameter** receives a *copy* of the value of the argument passed to it. As a result, if the function makes any changes to the parameter, this will not affect the argument. For example, in the following simple program,

```
#include <iostream>
```

```
void Foo (int num);
```

```
int main (void)
```

```
{int x = 10;
```

```
Foo(x);
```

```
cout << "x = " << x << '\n';
```

```
return 0;}
```

```
void Foo (int num)
```

```
{num = 0;
```

```
cout << "num = " << num << '\n';
```

```
}
```

- the single parameter of Foo is a value parameter. As far as this function is concerned, num behaves just like a local variable inside the function.
- When the function is called and x passed to it, num receives a copy of the value of x. As a result, although num is set to 0 by the function, this does not affect x.

The program produces the following output:

- num = 0;
- x = 10;

Reference parameter

- A **reference parameter**, on the other hand, receives the argument passed to it and works on it directly.
- Any change made by the function to a reference parameter is in effect directly applied to the argument.
- It is perfectly valid for a function to use pass-by-value for some of its parameters and pass-by-reference for others.
- The former is used much more often in practice.

Example2

We can rewrite the above example so that the value x is passed by reference.

```
#include <iostream>
using namespace std;
void Foo (int& num);
int main ()
{int x = 10;
Foo(x);
cout << "x = " << x << "\n";
return 0;}

void Foo (int& num)
{num = 0;
cout << "num = " << num << "\n";}
```

Example3

```
#include <iostream>

void getdatda(int& devnd, int& divisr) ;
void divide(int divdn, int divisr, int& quot, int& rem);
void print(int quot , int rem);

int main () // a function with out a parameter can be either left free or void as a parameter
{
int a,b,c,d;
getdata(a,b);
getdata(a,b,c,d);
print(c,d);
return 0;
}
```

```

void getdata(int& dividn, int& divisr)
{
    cout << "Enter two integers";
    cin >> dividn >> divisr;
    return;
}

void divide(int dividn, int divisr, int& quot, int& rem)
{
    quot = dividn/divisr;
    rem = dividn%divisr;
    return;
}

void print(int quot , int rem)
{
    cout << "Quotient is =:" << quot << endl;
    cout << "Remainder is = :" << rem << endl;
    return;
}

```


Global and Local Scope

- Everything defined at the program scope level (i.e., outside functions) is said to have a **global scope**.
- Thus the sample functions we have seen so far all have a global scope.
- Variables may also be defined at the global scope:

```

#include<iostream>
int year = 2021; // global variable
int Max (int, int); // global function
int main () // global function
{
int x,y;
cout<<"enter first number"<<endl;
cin>>x;
cout <<"enter the second number"<<endl;
cin>>y;
cout<<"year"<<year<<endl;
Max(x,y);
}
int Max(int x, int y)
{
    if( x>y)
        cout<<"the minimum is"y;
    else if (y<x)
        cout<<"the minimum is"x;
    else
        cout<<"they are equal";

cout<<"year"<<year<<endl;
}

```

- Un initialized global variables are automatically initialized to zero.

Since global entities are visible at the program level, they must also be unique at the program level.

- This means that the same global variable or function may not be *defined* more than once at the global level.
- Global entities are generally accessible everywhere in the program.
- Each block in a program defines a **local scope**. Thus the body of a function represents a local scope.
- The parameters of a function have the same scope as the function body.
- Variables defined within a local scope are visible to that scope only. Hence, a variable need only be unique within its own scope.
- Local scopes may be nested, in which case the inner scopes override the outer scopes.

For example, in

```
int x,y,z ; // xyz is global
void Foo (int x, int y, int z) // xyz is local to the body of Foo
{
    if (x> 0)
    {
        double x,y,z; // xyz is local to this block
        //...
    } }
```

Default Arguments

- Default argument is a programming convenience which removes the burden of having to specify argument values for all of a function's parameters.

For example, consider a function for reporting errors:

```
void Error (char message, int severity = 0);
```

- Here, severity has a default argument of 0; both the following calls are therefore valid:

```
Error( 'x', 3); // severity set to 3
```

```
Error( 'R'); // severity set to 0
```

- As the first call illustrates, a default argument may be overridden by explicitly specifying an argument.
- Default arguments are suitable for situations where certain (or all) function parameters frequently take the same values. In Error, for example, severity 0 errors are more common than others and therefore a good candidate for default argument. A less appropriate use of default arguments would be:

```
int Power (int base, unsigned int exponent = 1);
```

- Because 1 (or any other value) is unlikely to be a frequently-used one in this situation.
- A default argument need not necessarily be a constant. Arbitrary expressions can be used, so long as the variables used in the expression are available to the scope of the function definition (e.g., global variables).
- The accepted convention for default arguments is to specify them in function declarations, not function definitions.
- Because function declarations appear in header files, this enables the user of a function to have control over the default arguments. Thus different default arguments can be specified for different situations.
- It is, however, illegal to specify two different default arguments for the same function in a file.

Example

```
# include<iostream.h>
int devide (int a, int b=2);
int main()
{
cout<<devide(12);// Here the default value 2 is passed as a second argument.
cout<<endl;
cout<<devide(20,4);
return 0;
}
int devide (int a, int b)
{
int r;
r=a/b;
return (r);
}
```

Variable Number of Arguments / Overloaded functions

Two different functions can have the same name if the prototype e of their arguments are different. That means that you can give the same name to more than one function if they have either a different number of arguments or different types in their arguments.

Example

```
# include<iostream.h>
int devide (int a, int b=2);
int devide(int z, int r, int y);
float devide (float a, float b);
int main()
{int x=20, y=2;
float n=5.0, m=2.0;
cout<<devide(x,y);
cout<<endl;
cout<<devide(n,m);
cout<<endl;
cout<<devide(n,m,m);
cout<<endl;
return 0;
```



```
int devide (int a, int b)
{
return a/b;
}
int devide (int a, int b, int c)
{
int w=a/b
return w/c;
}
float devide (float x, float y)
{
return x/y;
}
```

- In this case we have defined two functions with the same name, but one of them accept two arguments of type **int** and the other accepts them of type **float** the compiler knows which one to call in each case by examining the type when the function is called.
- If it is called with two **ints** as an argument it calls to the function that has two **int** arguments in the prototype if it is called with two floats it will call to the one which has two **floats** in its prototype.
- For simplicity I have included the same code with both functions, but this is not compulsory. You can make two functions with the same name but with completely different behavior.

Recursion

- A function which calls itself is said to be **recursive**.
- Recursion is a general programming technique applicable to problems which can be defined in terms of themselves. Take the factorial problem, for instance, which is defined as:
 - Factorial of 0 is 1.
 - Factorial of a positive number n is n times the factorial of $n-1$.
- The second line clearly indicates that factorial is defined in terms of itself and hence can be expressed as a recursive function:

```
int Factorial (unsigned int n)
{
return n == 0 ? 1 : n * Factorial(n-1);
}
```

For n set to 3, the table below provides a trace of the calls to Factorial. The stack frames for these calls appear sequentially on the runtime stack, one after the other.